Haladó programozás II. C#

Beadandó feladat - Funkcionális Specifikáció

1. Bevezetés

Ez a dokumentum egy, a Neptunhoz hasonló oktatás-szervező rendszer backendjének funkcionális specifikációját tartalmazza. A rendszer az adminisztrátori oldalt kezeli, ahol lehet oktatókat, hallgatókat, tantárgyakat, kurzusokat kezelni, jegybeírást végezni.

A rendszer kizárólag backend funkcionalitásokat tartalmaz, a felhasználói interfész **Swagger-en keresztül** lesz elérhető. Tényleges bejelentkezés és autentikáció megvalósítás nem szükséges.

Leadási követelmények:

Connection string:

Az adatbázis eléréshez a connection string a következő legyen:

Server=(local);Database=CourseManagerDB_NEPTUN;Trusted_Connection=True;TrustServerCertificate=True;

A NEPTUN helyett mindenki a saját Neptun kódját használja.

Adatbázis kezdeti tartalma:

A tesztelés céljából a leadott kód mellé kérünk **VALÓSNAK TŰNŐ** teszt adatokat, amikkel az adatbázist a teszteléshez fel lehet tölteni. Ez lehet egy initdb.sql script, amit az adatbázison le lehet futtatni, vagy egy szerverbe épített szolgáltatás, ami észreveszi az üres adatbázist és feltölti adatokkal.

2. Technológiai környezet

- **Backend**: C# .NET 9
- **Adatbázis**: Microsoft SQL Server (MSSQL)
- **ORM**: Entity Framework Core
- **API dokumentáció**: Swagger

3. Főbb Funkcionalitások

3.1 Felhasználókezelés

A rendszer több felhasználós, így minden felhasználó egyedi azonosítóval, névvel és e-mail címmel rendelkezik. Lehet felhasználókat regisztrálni és jelszót módosítani. (Egy ilyen rendszerbe a felhasználók nem maguk regisztrálnak, hanem csak az ügyintézők tudják őket felvenni.) A tényleges bejelentkezés funkció nem szükséges, úgy tekintjük, mintha minden műveletet egy adminisztrátor végezne.

Minden felhasználónak legyen továbbá egy típusa, ami meghatározza a számára elérhető lehetőségeket. Három felhasználó típust kezeljen a rendszer: hallgató, oktató, ügyintéző. Hallgató esetén fontos tárolni, hogy nappalis, vagy levelezős. Feltehetjük, hogy minden felhasználó pontosan az egyik típusba tartozik. Az egyszerűség kedvéért a felhasználó típusát létrehozáskor megadjuk, és utána nem változik.

CRUD műveletek:

- **Felhasználó regisztráció** (POST /api/users/register)
    - **Input**: { username, email, password, ... }
    - **Validálás**: Az e-mail címnek egyedinek kell lennie.
- **Felhasználó adatainak lekérdezése** (GET /api/users/{userId})
- **Felhasználó adatainak frissítése** (PUT /api/users/{userId})
    - A felhasználó típusa nem módosítható

3.2 Felhasználó inaktívvá tétele

A rendszer komplex összefüggései miatt a felhasználókat nem töröljük. Helyette, ha egy felhasználóra már nincs szükség, akkor inaktívvá kell tenni. Ekkor minden korábbi, vele végzett művelet maradjon a rendszerben, de új műveletet ne lehessen vele végezni. Később igény szerint újra aktiválható a profil.

Műveletek:

- **Felhasználó inaktiválása** (POST /api/users/{userId}/deactivate)
- **Felhasználó újra aktiválása** (POST /api/users/{userId}/reactivate)

3.3 Tantárgyak kezelése

A rendszer kezelje az intézmény által hirdetett tantárgyakat. A tárgyaknak mindenképp legyen kódja, neve, valamint kredit értéke. A tantárgyakat sem töröljük, csak inaktívvá tesszük. A korábbi adatok megmaradnak, de inaktív tárgyból már nem hirdethetőek kurzusok.

CRUD műveletek:

- **Tárgyak listázása** (GET /api/subjects)
- **Egy adott tantárgy adatainak lekérdezése** (GET /api/subjects/{subjectId})
- **Új tantárgy hozzáadása** (POST /api/subjects)
    - **Input**: { code, name, credits }
- **Tárgy adatok frissítése** (PUT /api/subjects/{subjectId})
- **Tantárgy inaktiválása** (POST /api/subjects/{subjectId}/deactivate)
- **Tantárgy újra aktiválása** (POST /api/subjects/{subjectId}/reactivate)

3.4 Kurzusok kezelése

A tárgyak meghirdetése kurzusokon keresztül történik. Csak aktív tárgyhoz lehet kurzust hirdetni. A kurzusnál meg kell adni a saját kódját, a tárgyat, a félévet, maximum létszámot, a kurzus típusát (elmélet, gyakorlat, labor), a képzés formáját (nappali, levelező), valamint az óraszámot. Az óraszám lehet heti, vagy féléves (tipikusan attól függ, hogy nappalis vagy levelező tagozatra hirdetünk, de nappalin is előfordulnak tömbösített órák féléves óraszámmal). A kurzusokhoz oktatót is kell rendelni (lehet több is). Az egyszerűség kedvéért ezt létrehozáskor adjuk meg, utána nem változik.

Műveletek:

- **Kurzus hirdetése** (POST /api/courses)
    - **Input**: { courseCode, subjectCode, semester, maxStudents, type, form, ... }
        - A *form* a nappali (full-time) vagy levelező (part-time), vagy lehet tagozat nélküli is (nappalis és levelezőn egyben).
        - Az óraszám és az oktató(k) átadását is meg kell még oldani (ez reprezentáció függő)
- **Kurzus adatok lekérése** (GET /api/courses/{courseId})
- **Kurzus adatok frissítése** (PUT /api/courses/{courseId})
- **Kurzus törlése** (DELETE /api/courses/{courseId})
    - A kurzus ténylegesen törölhető
    - **Validálás:** A kurzus csak akkor törölhető, ha nincs rajta hallgató

3.5 Hallgatók és kurzusok

A hallgatók azáltal tudják felvenni a tárgyakat, hogy a kurzusokra jelentkeznek. Itt több szabályt is be kell tartani:

- A hallgató tárgyat vesz fel azáltal, hogy megadja, azon belül mely kurzusokat veszi fel. Minden, a félévben és képzési módon elérhető kurzusból pontosan egyet kell felvenni.
- Nappalis hallgató csak nappalis, levelezős hallgató csak levelezős kurzust vehet fel (tagozat nélküli kurzust bárki felvehet).
- Az összes felvett kurzus ugyanahhoz a félévhez tartozzon.
- A kurzuson nem lehet több hallgató, mint amennyit a létszám megenged.

Műveletek:

- **Tárgyra feliratkozás** (POST /api/subjects/{subjectId}/register)
    - **Input**: { studentId, courseIds }
    - **Validálás**: A fent leírtak szerint
- **Tárgyról lejelentkezés** (POST /api/subjects/{subjectId}/unregister)
    - **Input**: { studentId, semester }
- **Kurzus átjelentkezés** (POST /api/courses/change)

    - **Input**: { studentId, fromCourseId, toCourseId }
    - **Validálás**: Csak ugyanazon tárgyon belül ugyanolyan típusú kurzusra átjelentkezni. Az összes fenti szabálynak továbbra is teljesülnie kell

- **Kurzus hallgatóinak listázása** (GET /api/courses/{courseId}/students)
- **Tárgy hallgatóinak listázása adott félévre** (GET /api/subjects/{subjectId}/students)
    - **Input:** { semester }

3.6 Órarend

A kurzusokhoz szükséges tárolni az órarendi időpontokat is. A heti rendszerességű órák 14 héten keresztül mindig ugyanakkor vannak, míg a tömbösített órák több, egymástól független időponttal rendelkeznek.

Műveletek:

- **Órarendi időpont(ok) megadása** (POST /api/courses/{courseId}/schedule)
    - A Swagger oldalon legyen egyértelműen leírva, példával együtt, hogy milyen módon kell megadni az időpontokat (milyen adatokkal, milyen formában)
- **Órarendi időpont(ok) módosítása** (POST /api/courses/{courseId}/schedule/modify)
    - A Swagger oldalon legyen egyértelműen leírva, példával együtt, hogy milyen módon kell módosítani az időpontokat (milyen adatokkal, milyen formában)
- **Értesítés**:
    - Egy időzített háttérszolgáltatás (Background Service) rendszeresen figyelje a kurzusok órarendi adatait. Minden kurzus előtt 30 perccel generáljon egy értesítést minden, a kurzuson lévő hallgatónak és oktatónak.
    - Üzenetküldés híján a generált értesítéseket az adatbázisba kell logolni, hogy utólag lekérdezhetőek legyenek (legyen benne minden infó, ami alapján látható, hogy kinek lett volna elküldve és milyen okból).
- **Minden generált értesítés listázása** (GET /api/notifactions)
    - **Input:** Lehessen szűrni felhasználóra és/vagy kurzusra